



Middle East Technical University Northern Cyprus Campus  
Computer Engineering Program

CNG 492 Computer Engineering Design II

**DIP**  
**Discovering Issue-tracking Profiles of Software  
Development Teams**

2079242 İlbey Evcil  
2315315 Burak Kaan Kahraman  
2243384 Muhammed Didin  
2243442 Çağatay Gültekin

Supervised by  
Dr. Sukru Eraslan

**Final Report**

# Contents

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                               | <b>4</b>  |
| 1.1      | Problem Formulation and Background Information . . . . .          | 4         |
| 1.2      | Aims and Objectives . . . . .                                     | 4         |
| 1.3      | Stakeholders . . . . .                                            | 5         |
| <b>2</b> | <b>Identification of Constraints</b>                              | <b>6</b>  |
| 2.0.1    | Economical Constraints . . . . .                                  | 6         |
| 2.0.2    | Ethical Constraints . . . . .                                     | 6         |
| <b>3</b> | <b>Literature Review</b>                                          | <b>7</b>  |
| <b>4</b> | <b>Research Method</b>                                            | <b>9</b>  |
| <b>5</b> | <b>Requirements Specifications</b>                                | <b>10</b> |
| 5.1      | Functional Requirements . . . . .                                 | 10        |
| 5.2      | Non-Functional Requirements . . . . .                             | 11        |
| 5.3      | Assumptions and Limitations . . . . .                             | 11        |
| 5.4      | Structured Use Case Diagram . . . . .                             | 12        |
| 5.5      | High Level Sequence Diagram . . . . .                             | 13        |
| 5.6      | External Interfaces . . . . .                                     | 14        |
| 5.7      | (Graphical) User Interface . . . . .                              | 14        |
| <b>6</b> | <b>System Architecture and Models</b>                             | <b>19</b> |
| 6.1      | Architectural Model . . . . .                                     | 19        |
| 6.2      | Process Model . . . . .                                           | 20        |
| 6.3      | Data Flow Models . . . . .                                        | 20        |
| 6.4      | Class Diagram . . . . .                                           | 22        |
| 6.5      | List of Special Data Structures and Required Data Types . . . . . | 22        |
| 6.6      | List of Special Algorithms . . . . .                              | 23        |
| <b>7</b> | <b>Project Management</b>                                         | <b>30</b> |
| 7.1      | Software Estimations . . . . .                                    | 30        |
| 7.1.1    | Compute Unadjusted Total of Function Points (UTFP) . . . . .      | 30        |
| 7.1.2    | Computer Value adjustment factor (VAF) . . . . .                  | 31        |
| 7.1.3    | Compute Adjusted Total of Function Points (ATFP) . . . . .        | 31        |
| 7.1.4    | Compute Estimated Lines of Code (LOC) . . . . .                   | 31        |
| 7.1.5    | Total Development Time(TDEV) . . . . .                            | 32        |
| 7.1.6    | Rough Schedule Estimate . . . . .                                 | 32        |
| 7.1.7    | Efficient Schedule . . . . .                                      | 32        |

|           |                                                       |           |
|-----------|-------------------------------------------------------|-----------|
| 7.1.8     | Estimated Team Size . . . . .                         | 33        |
| 7.2       | Milestones, Durations and Task Allocations . . . . .  | 34        |
| 7.3       | Gantt Chart . . . . .                                 | 35        |
| <b>8</b>  | <b>Software Implementation</b>                        | <b>36</b> |
| 8.1       | List of Libraries . . . . .                           | 36        |
| 8.1.1     | Tkinter . . . . .                                     | 36        |
| 8.1.2     | PyGithub . . . . .                                    | 36        |
| 8.1.3     | matplotlib . . . . .                                  | 36        |
| 8.1.4     | csv . . . . .                                         | 36        |
| 8.1.5     | pickle . . . . .                                      | 37        |
| <b>9</b>  | <b>Software Testing</b>                               | <b>38</b> |
| 9.1       | Testing Strategy . . . . .                            | 38        |
| 9.2       | Unit Testing . . . . .                                | 39        |
| 9.2.1     | Graphical User Interface . . . . .                    | 39        |
| 9.2.2     | Metric Calculations . . . . .                         | 39        |
| 9.3       | Exporting Data to .csv Format . . . . .               | 42        |
| 9.3.1     | Participants . . . . .                                | 42        |
| 9.3.2     | Pass/Fail Criteria . . . . .                          | 42        |
| 9.3.3     | Methodology . . . . .                                 | 42        |
| 9.4       | Exporting Data as Graphical Representations . . . . . | 42        |
| 9.4.1     | Participants . . . . .                                | 42        |
| 9.4.2     | Pass/Fail Criteria . . . . .                          | 42        |
| 9.4.3     | Methodology . . . . .                                 | 43        |
| 9.5       | Answers to the questions . . . . .                    | 43        |
| 9.5.1     | Participants . . . . .                                | 43        |
| 9.5.2     | Pass/Fail Criteria . . . . .                          | 43        |
| 9.5.3     | Methodology . . . . .                                 | 44        |
| 9.6       | System and Integration Testing . . . . .              | 46        |
| 9.6.1     | Participants . . . . .                                | 46        |
| 9.7       | Integration Testing . . . . .                         | 46        |
| 9.7.1     | System Testing . . . . .                              | 46        |
| 9.8       | Performance Load and Stress Testing . . . . .         | 46        |
| 9.8.1     | Participants . . . . .                                | 46        |
| 9.8.2     | Performance Load Testing . . . . .                    | 46        |
| 9.8.3     | Stress Testing . . . . .                              | 49        |
| <b>10</b> | <b>Conclusion</b>                                     | <b>51</b> |
| 10.1      | Example Research Results . . . . .                    | 52        |

# List of Figures

|      |                                                            |    |
|------|------------------------------------------------------------|----|
| 5.1  | Use Case Diagram . . . . .                                 | 12 |
| 5.2  | High Level Sequence Diagram . . . . .                      | 13 |
| 5.3  | External Interfaces . . . . .                              | 14 |
| 5.4  | DIP Research Tool Main Window . . . . .                    | 15 |
| 5.5  | Get Repo Window . . . . .                                  | 15 |
| 5.6  | Metric Selection Window . . . . .                          | 16 |
| 5.7  | Question Selection Window . . . . .                        | 16 |
| 5.8  | Result Page for Metric Calculations . . . . .              | 17 |
| 5.9  | Result Page for Question Calculations . . . . .            | 18 |
| 6.1  | Architectural Model . . . . .                              | 19 |
| 6.2  | Process Model . . . . .                                    | 20 |
| 6.3  | Data Flow Model (Level 0) . . . . .                        | 20 |
| 6.4  | Data Flow Model (Level 1) . . . . .                        | 21 |
| 6.5  | Data Flow Model (Level 2) . . . . .                        | 21 |
| 6.6  | Class Diagram . . . . .                                    | 22 |
| 7.1  | Efficient Schedules . . . . .                              | 33 |
| 7.2  | Gantt Chart . . . . .                                      | 35 |
| 9.1  | DIP-Group/GithubTracker repository issues page . . . . .   | 38 |
| 9.2  | Unit Test Conditions for Metric Calculations . . . . .     | 40 |
| 9.3  | Example from script used in metric calculations . . . . .  | 41 |
| 9.4  | Example from script used in metric calculations . . . . .  | 41 |
| 9.5  | Unit Test Conditions for Answer to the Questions . . . . . | 44 |
| 9.6  | Question test function from script used in test . . . . .  | 45 |
| 9.7  | Main function from script used in test . . . . .           | 45 |
| 9.8  | Performance Test Results . . . . .                         | 48 |
| 9.9  | Performance Test Results . . . . .                         | 49 |
| 9.10 | Stress Test Results . . . . .                              | 50 |
| 10.1 | Repository Names . . . . .                                 | 52 |
| 10.2 | Metric Names . . . . .                                     | 52 |
| 10.3 | Metrics Result Table . . . . .                             | 53 |
| 10.4 | Question Names . . . . .                                   | 53 |
| 10.5 | Question Results Table . . . . .                           | 53 |

# Chapter 1

## Introduction

### 1.1 Problem Formulation and Background Information

Software development platforms (such as GitHub, GitLab, etc.) are widely used by software developers for collaborative working. These platforms do not only allow to use a version control system (VCS) but also offer additional features to support software engineering activities, such as issue-tracking. Issue tracking allows software developers to keep track of tasks and bugs in their projects.

Issue-Tracking enables an organized follow-up of the issues determined in the projects, and the regular execution of the project by determining who should do which issues. In addition, these issues can be put into specific milestones and the tasks determined within a certain period can be easily followed. And using these data, different metrics can be created. By using these metrics, decisions can be made to create a specific profile and make the project more efficient.

Software development teams can reduce their efficiency in the project by applying this issue tracking system in a disorganized way. It is not known how to use this platform in the most appropriate and efficient way. In addition, the fact that dispersed issue tracking system data cannot be easily accessed, brought together and regularity cannot be achieved by the researchers.

There is no information on how to use the issue-tracking systems on the platforms correctly and efficiently. In addition, this system also does not have any standard. For this reason, it is necessary to determine what people should pay attention to on this platform and what they should use for what. We are trying to develop a tool to reduce this deficiency and help build best practice. Best practice can be developed in line with the data obtained by examining the repository determined using this tool.

As a result of this project, we create a tool which enables researchers to find a profile as the best practices in using issue-tracking, according to the issue-related metrics and questions we have determined.

### 1.2 Aims and Objectives

The aims of this project are as follows:

- To create a tool which is used to guide for learners, small companies and those who plan to use the issue-tracking system efficiently in their projects.

- To show that issue related metrics are important and significant data in terms of business efficiency and employee productivity.

The objectives are as follows:

- extract issue-related data from a software development platform.
- process the data to generate a set of issue-related metrics.
- process the data to generate a set of issue-related questions.
- discover how software development teams use issue-tracking during the collaborative software development.

## **1.3 Stakeholders**

- Project Managers
- Professional Software Developers
- Learners
- Teaching Stuff
- Researchers

# Chapter 2

## Identification of Constraints

### 2.0.1 Economical Constraints

While we were doing our project, we had to develop our application using a free account on github, as our group did not have sufficient economic resources to be used in development. However, it is possible to increase our current performance by 25 times by paying the necessary fee and obtaining the accounts determined on github. In other words, transactions can be made on large repos without any limits, within the payment of the required fee. In addition, if necessary, smaller size memberships can be purchased and adjusted according to the scale of the project to be used. In fact, this situation also includes the feature of being scaleable according to the project.

### 2.0.2 Ethical Constraints

In order to comply with the ethical rules while carrying out our project, we conducted research on public repository only. Although we have done research on public repositories, as we have stated in economic constructs, there is a possibility to conduct research on private repository by using a different premium membership apart from the free account. Therefore, if the person who will use the tool has a premium account, they can also use Github's feature of extracting data from private accounts, which is provided to premium members.

# Chapter 3

## Literature Review

Software developers use version control systems for collaborative coding. These systems are integrated into several software development platforms (including GitLab and GitHub), which provide additional software engineering functionalities such as issue-tracking, milestone-tracking, and time-tracking etc. Due to these functionalities, they are attracted by various companies and scientists. In the literature, many review papers summarize all the functionalities or the research directions such as [1], [2], [3], [4], [5]. Features of software development platforms include allowing software developers to create and resolve issues (items of work 'to do'). These platforms also allow researchers to collect data about how the programmers interact with these platforms and the work they are doing. In [4], it is stated that the data related to the issues can provide essential and meaningful information about the repository on issues such as communication between team, programming characteristics, and management style. From this perspective, examining software development platforms' data through issue tracking systems will lead us to significant results. In [1], this idea was supported by emphasizing that the project participants thought that meaningful data could be produced with the data using issue tracking. In [2], it is mentioned that process data archived in an Issue Tracking System as an event log can be mined to track the bug-resolving process of a project and can also be used for contribution and performance assessment of individuals. Researchers in [2] investigate opening and closing trends for all projects and the similarities and differences in their characteristics. They propose a metric called "Bug Fixing Score", as shown in Equation 1, to quantify the bugs opening and closing trend of a project. The metric can serve as an indicator of the bug fixing performance efficiency.

$$BugFixingScore = 1/(n - i) \times \sum_{i=1}^{i=n} (log$$

Equation 1: Bug Fixing Score

Where BO = Bugs opened until time instant i, BC = Bugs closed until time instant i, "i" is the first time instant when BO > 0 and "n" is the maximum recorded time instant.

By creating a metric like this, we plan to make observations by scoring the issue solving situations of repository contributors.

In [3], it is mentioned that the opening and closing time of the issues is important and that the connection between them will create meaningful data on the total open time of an issue. It is also mentioned that the information about how long the issues that have never been closed have been open is also important. For these reasons, we will reach the opening date and total open time of the open issues, the date when the closed issues were opened and closed, and then how long they were open.



In [5], it is emphasized that the number of issues per student, the number of issues assigned per student, the number of issue comments per student, and the average issue solving time per student are important metrics. When this situation is taken into consideration, we will try to obtain meaningful results by reaching these metrics and using the data we have obtained.

# Chapter 4

## Research Method

The appropriate research methods that we have identified during the construction phase of this project are as follows:

- This project will be developed in specific sprints using SCRUM. Scrum development model is chosen because the project will be divided into sprints and changes can be made with the advice of the consultant academician.
- Development will be done in a modular way. Issue related metrics will be properly distributed among our team members. We chose modular development in order to ensure the readability and optimization of the code where 4 people were working on the project.
- This pipeline will be tested with various open-source repositories on GitHub including the repositories of reputable software development companies and software engineering learners.
- We will create a testing repository so we can do our program tests with it for the first time.
- In order to obtain Issue Related Metrics, we will use some mathematical coefficients and formulas previously created, such as Gini Coefficient, bug fixing score, and so on.
- External repositories will be used for testing.
- We determined our metrics and questions by considering the differences of good issue management, by doing source research, and at the level provided by the pyGithub library and GitHub.

# Chapter 5

## Requirements Specifications

### 5.1 Functional Requirements

1. The system shall communicate with the GitHub Platform.
  - (a) The system shall get URL link from the user.
  - (b) The system shall communicate with the GitHub using API.
2. The system shall retrieve issue-related data from a repo.
  - (a) The system shall get required data as .pickle format like repo's owner name and repo's name from given URL.
  - (b) The system shall get related data from specified address.
  - (c) The system shall get all the issue related data to be used from this specific repo including issues open and close time, label type, number of active issues, number of closed issues, etc.
3. The system shall process on the retrieved data by using specified metrics and questions.
  - (a) The system shall retrieve issues that match metrics and questions specified by the user.
  - (b) User shall enter the start and end date for the specific issues.
  - (c) The system shall process the retrieved data to create metrics and questions.
4. The system shall arrange and put into report form from interpreted data.
  - (a) The system shall group these processed data and related metrics repo by repo as a row for each repo.
  - (b) The system shall give the details of the processed information in a table format.
  - (c) The system shall give the details of the each repo in a one Row.
5. The system shall visualise the arranged and reported data as graph or chart form.
  - (a) The system shall calculate chart or graph according to these arranged data.
  - (b) The system shall render a chart of the selected type as an image.
6. The system shall export the specific processed issue related metric details to the user.

## 5.2 Non-Functional Requirements

1. The system should be available all the time because users will use this tool whenever they want to do research.
2. The system should be easy to use because new learners may want to use.
  - (a) The system should have a simple and easy to use UI.
  - (b) The system should have a Information pop-ups about how to use app.
3. The system should provide output in the most used formats because the outputs of this tool can be used in most of the applications.
  - (a) The system should give the specific processed issue related metric details in .csv format

## 5.3 Assumptions and Limitations

1. We assumed that this tool will be an open source, free and accessible program.
2. We assumed that the program will only run on public repositories.
3. We assumed that people such as Software developers, Project Managers and Researchers could use the program other than us.
4. We assumed that this application would search based on URL links and specified date ranges, and that would be an easier method.
5. The API rate limitation causes a limitation for the system to run in many repositories and issues because GitHub allows us to use 5000 requests per an hour.

## 5.4 Structured Use Case Diagram

Figure 5.1 shows that the relationship between the user and the different use cases of our system in which the user is involved. In this diagram, user represents project managers, researchers etc. This diagram also shows how we fetch raw data from GitHub and how we process these data.

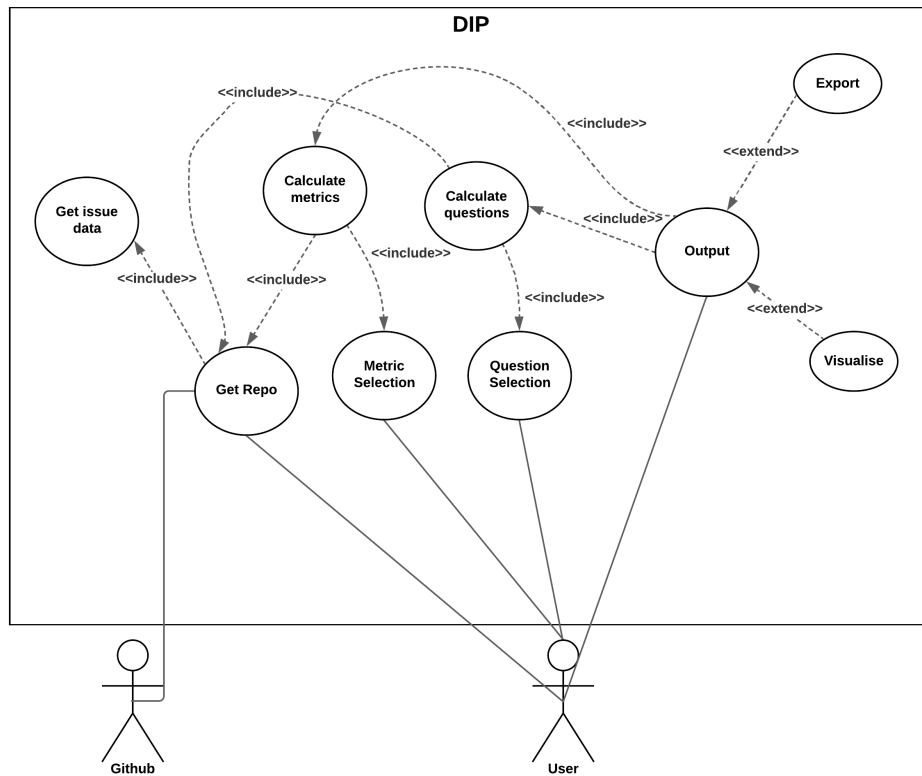


Figure 5.1: Use Case Diagram

## 5.5 High Level Sequence Diagram

Figure 5.2 shows object interactions of our tool arranged the stages of creating and displaying metrics chosen by the user according to the time sequence.

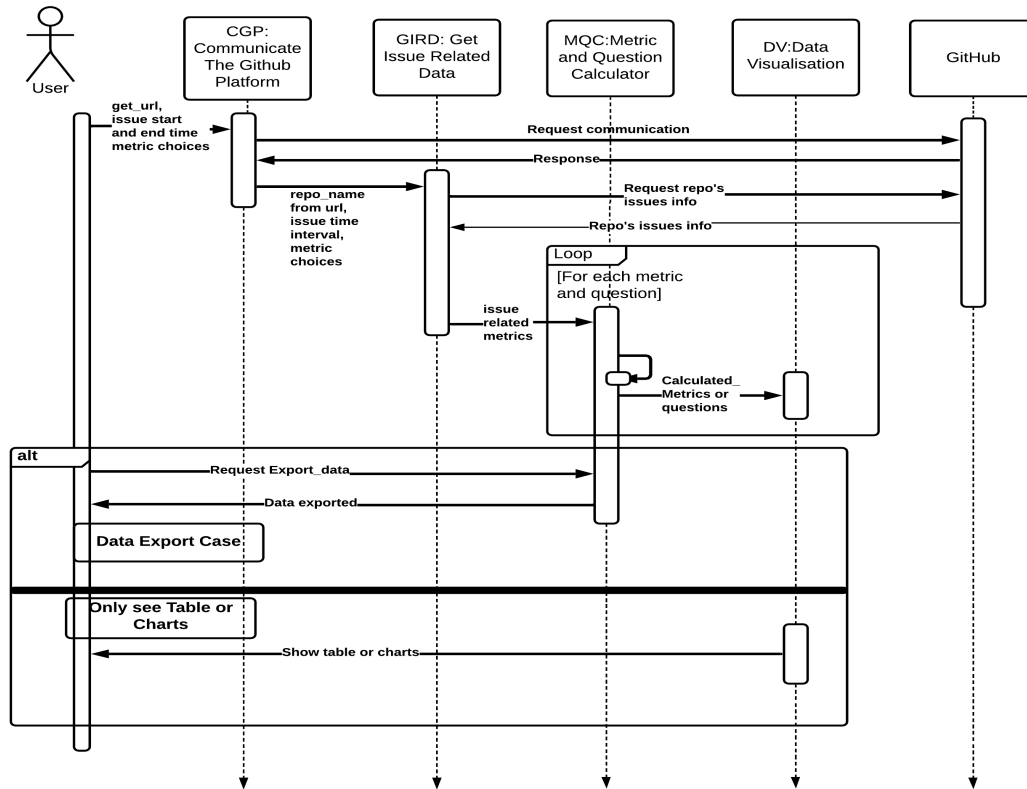


Figure 5.2: High Level Sequence Diagram

## 5.6 External Interfaces

Discovering Issue-tracking Profiles of Software Development Teams(DIP) provides a facility to effect data exchanges and transfers with external systems. You can export issue-related data from an external system such as GitHub. The external system is a GitHub repository given via a link. The data exchange can be effected through an interface with the external system. This interface consists of the following components:

- An interface definition that will capture all the information that is needed for processing and effecting the data exchange.
- You can designate all the metrics and questions that need to be read from GitHub.

Figure 5.3 shows that relations between our system and the external system.

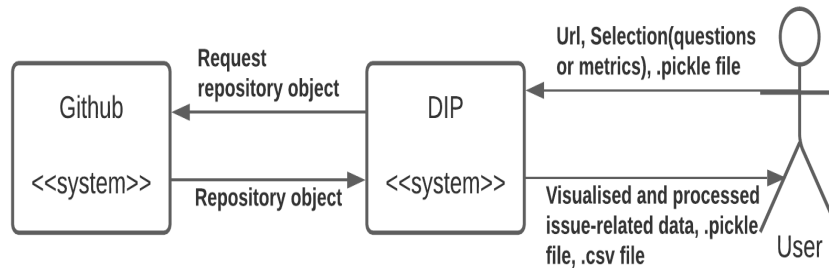


Figure 5.3: External Interfaces

## 5.7 (Graphical) User Interface

Figure 5.4 is The main page of the tool. There are two options. Get New Repo is for taking new repo from GitHub. Load New Repo And Continue is for Loading existing pickle file of the repository.

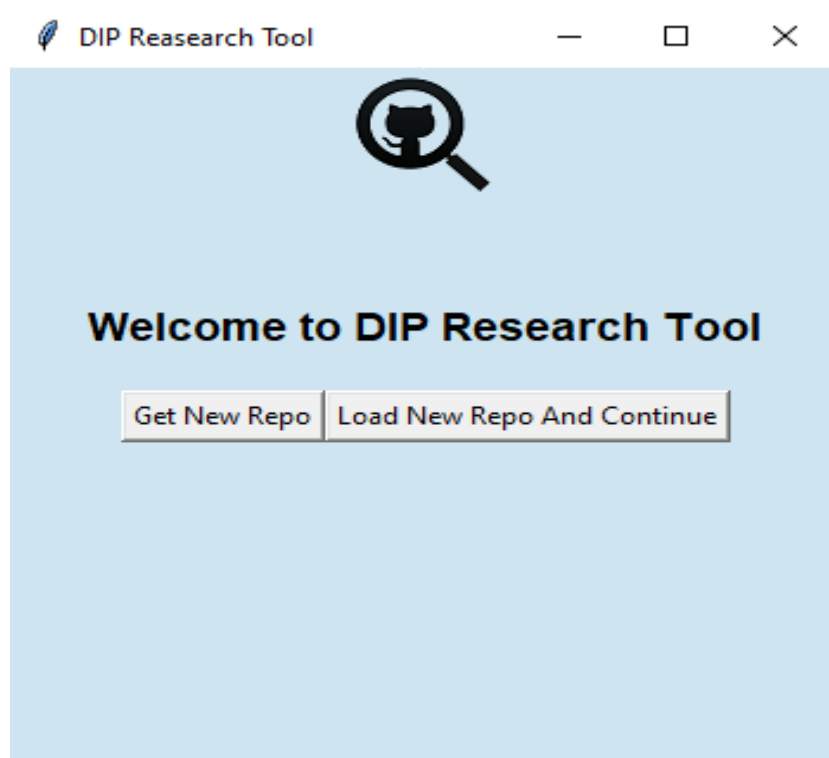


Figure 5.4: DIP Research Tool Main Window

Figure 5.5 shows Get Repo window. This window is for getting new Repo from GitHub using URL of the repo.

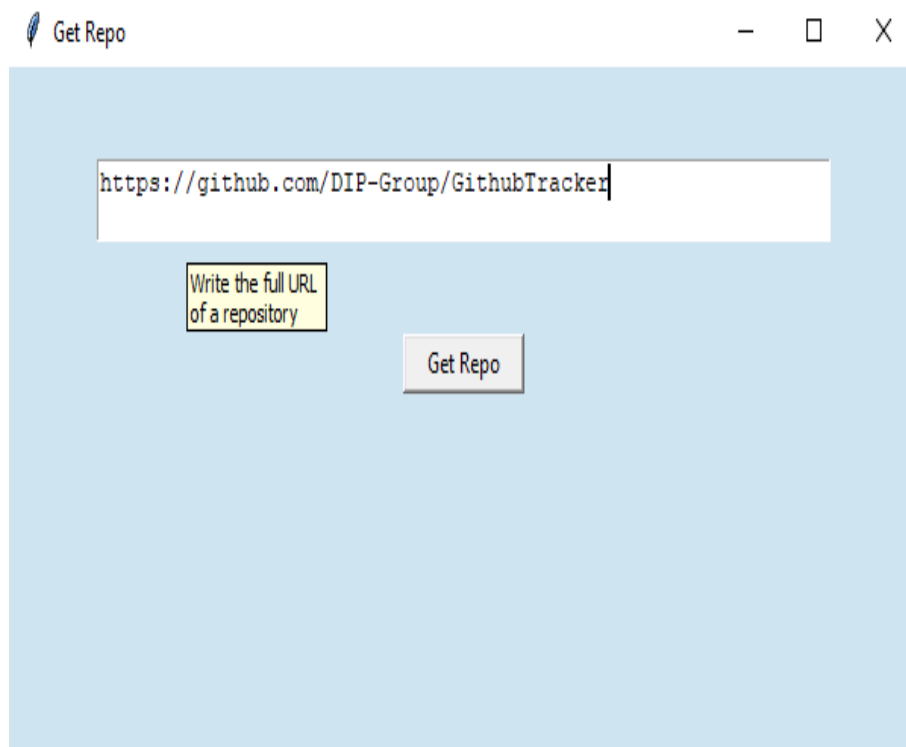


Figure 5.5: Get Repo Window

Figure 5.6 shows that metric selection page that user select metrics that wants to be



calculated.

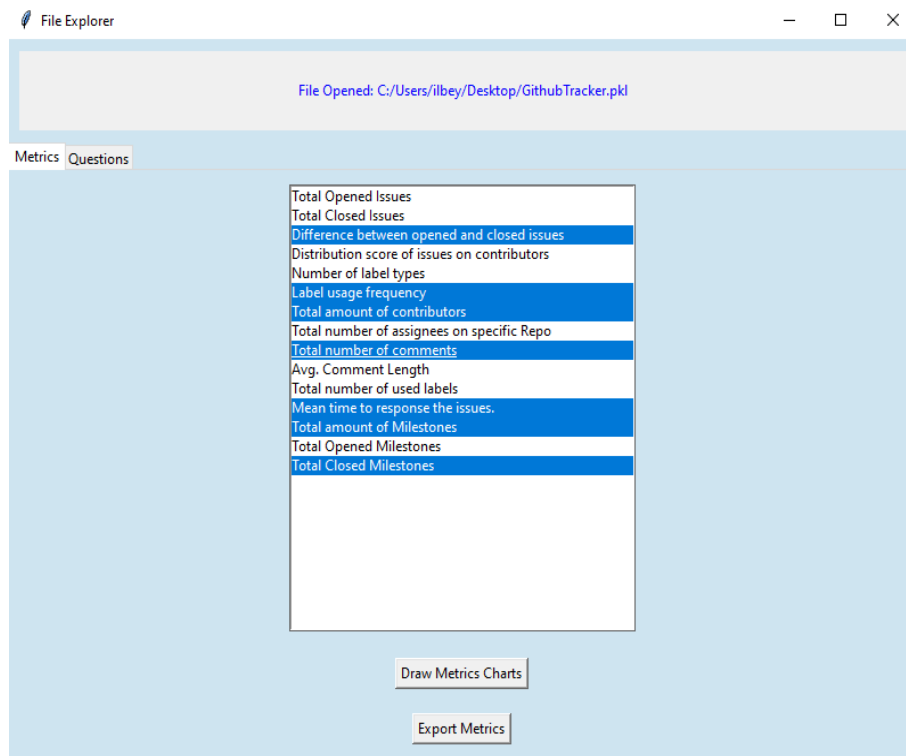


Figure 5.6: Metric Selection Window

Figure 5.7 shows that questions selection page that user select questions that wants to be calculated.

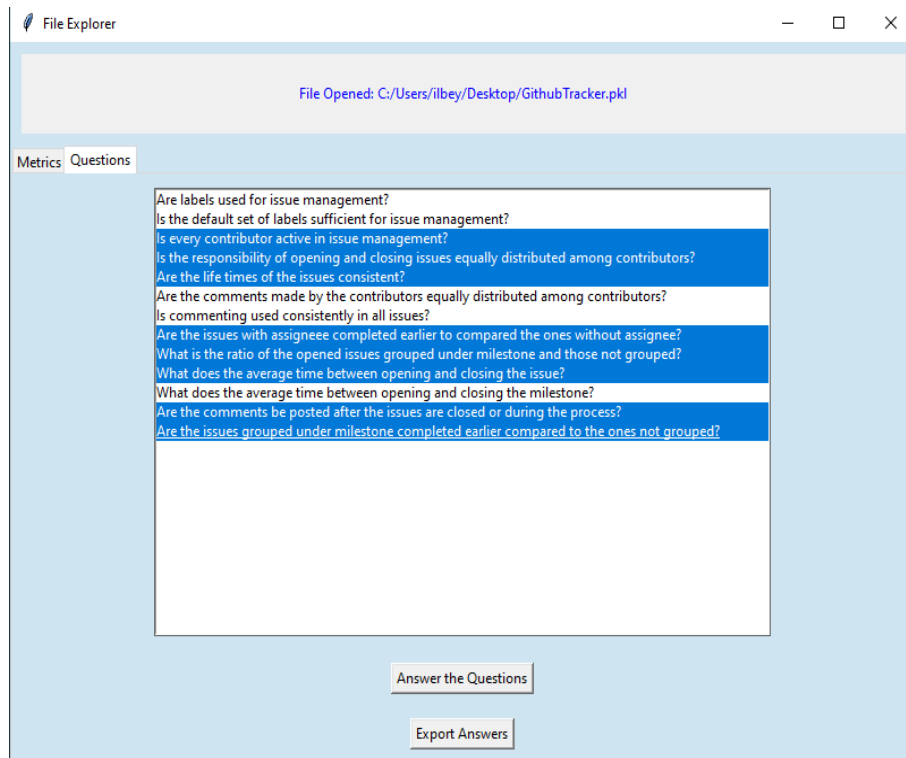


Figure 5.7: Question Selection Window

Figure 5.8 shows that metric calculation result page. Metric calculations can be as a graph or a text depend on metric that calculated.

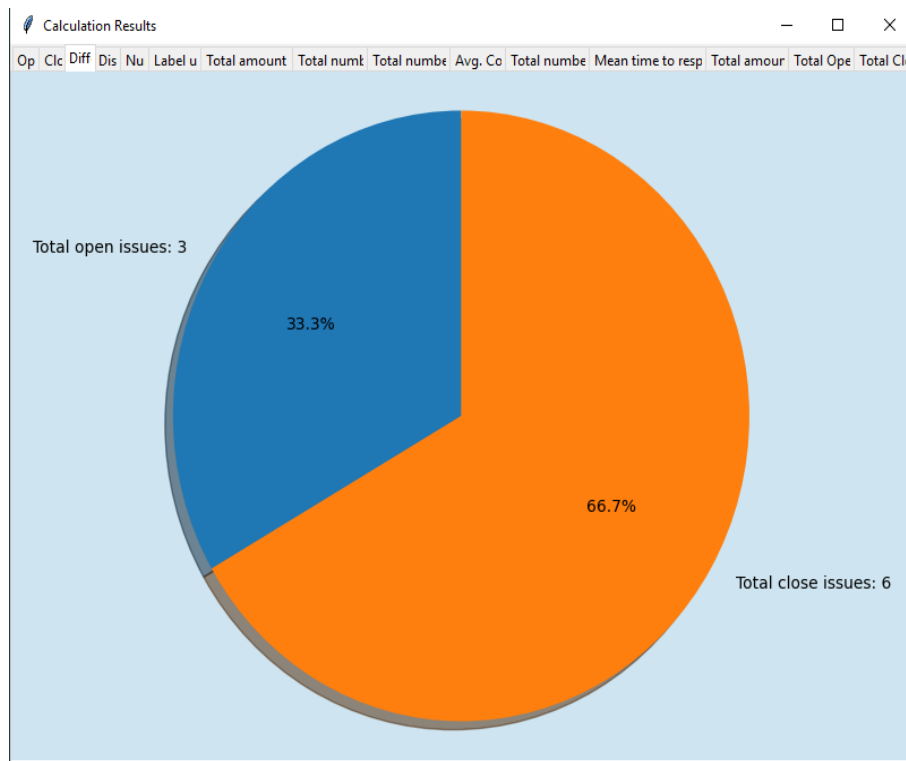


Figure 5.8: Result Page for Metric Calculations

Figure 5.9 shows that Question and Answer table that shows the questions that user chose and related answers.

| Questions                                                                       | Answers |
|---------------------------------------------------------------------------------|---------|
| Are labels used for issue management?                                           | No      |
| Is the default set of labels sufficient for issue management?                   | Yes     |
| Is every contributor active in issue management?                                | No      |
| Is the responsibility of opening and closing issues equally distributed among c | No      |
| Are the life times of the issues consistent?                                    | No      |
| Are the comments made by the contributors equally distributed among contri      | No      |
| Is commenting used consistently in all issues?                                  | No      |
| Are the issues with assignee completed earlier to compared the ones without     | No      |
| What is the ratio of the opened issues grouped under milestone and those not    | 100     |
| What does the average time between opening and closing the issue?               | 36.0    |
| What does the average time between opening and closing the milestone?           | 7.0     |
| Are the comments be posted after the issues are closed or during the process?   | No      |
| Are the issues grouped under milestone completed earlier compared to the or     | Yes     |

Figure 5.9: Result Page for Question Calculations

# Chapter 6

## System Architecture and Models

### 6.1 Architectural Model

Figure 6.1 shows that is event-driven architecture model which is like viewpoint of our system.

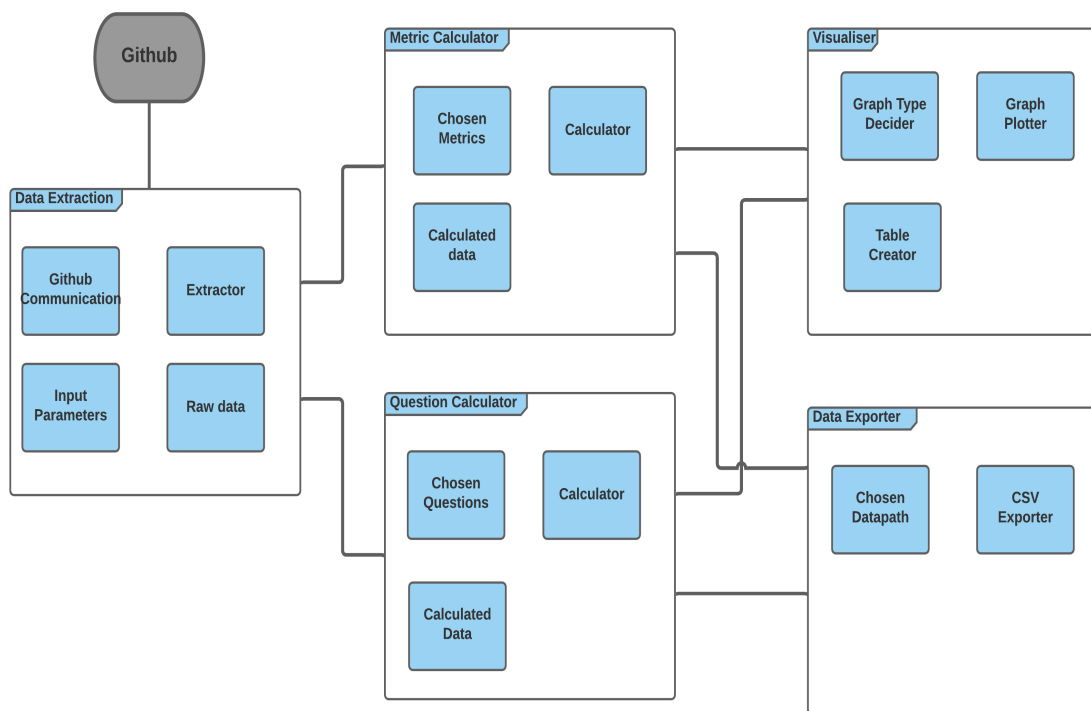


Figure 6.1: Architectural Model

## 6.2 Process Model

Figure 6.2 shows a set of related activities of our software system. The model that shows the realization and sequential selection stages of the events in our program.

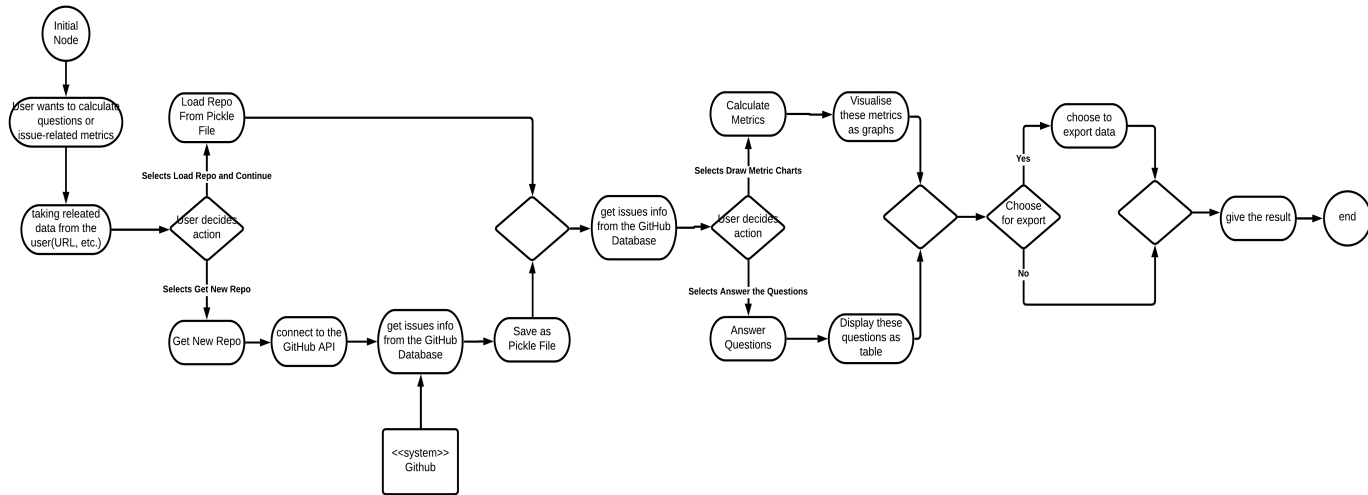


Figure 6.2: Process Model

## 6.3 Data Flow Models

Figure 6.3 maps out the flow of information for our processes and system.

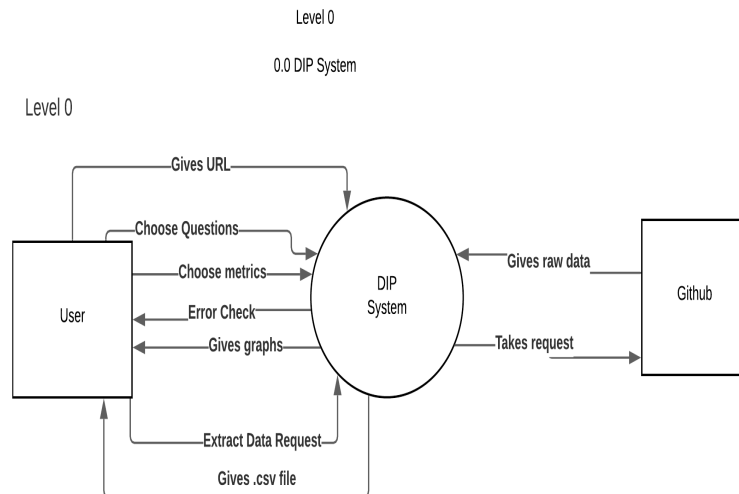


Figure 6.3: Data Flow Model (Level 0)

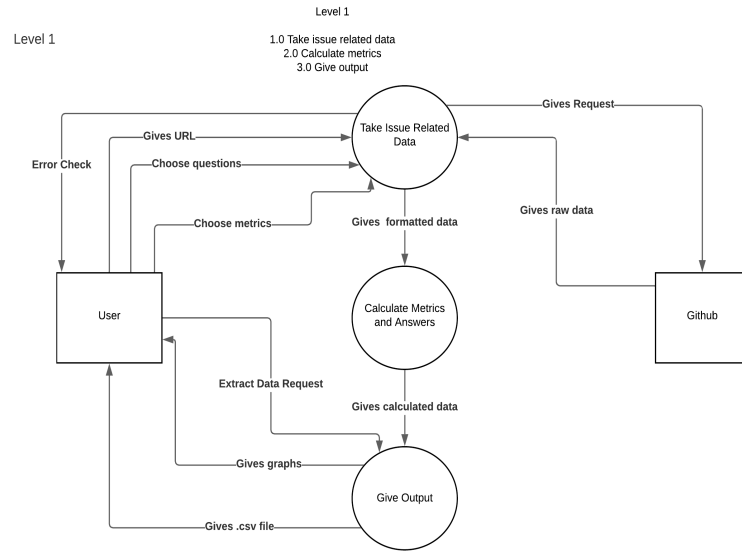


Figure 6.4: Data Flow Model (Level 1)

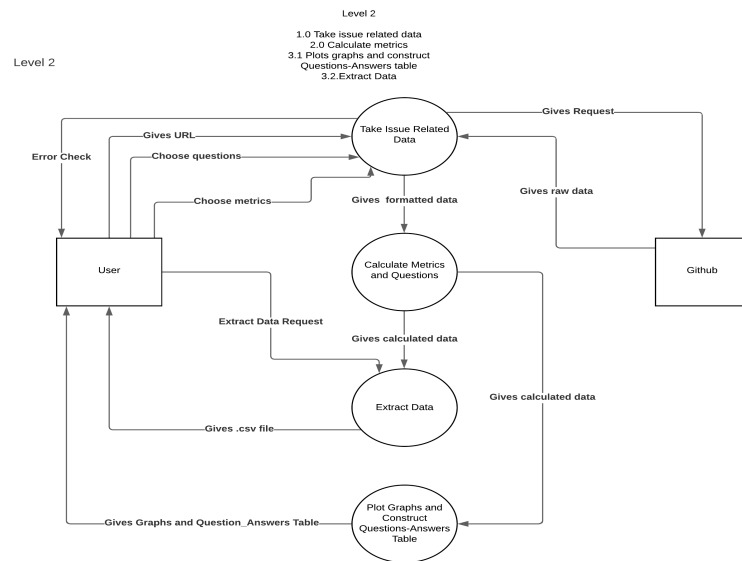


Figure 6.5: Data Flow Model (Level 2)

## 6.4 Class Diagram

Figure 6.6 shows that class relations

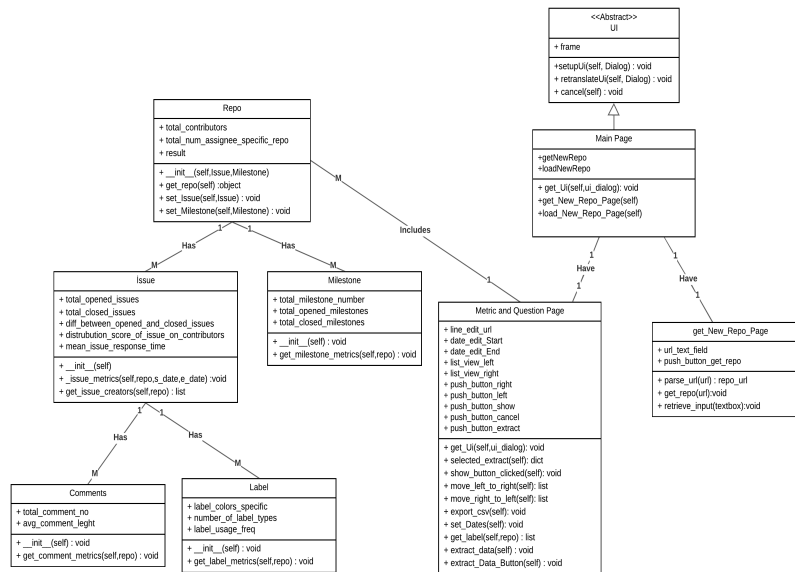


Figure 6.6: Class Diagram

## 6.5 List of Special Data Structures and Required Data Types

Special Data Structures and Required Data Types:

- Array
- Dictionary
- List
- Integer
- Floating-point number
- Character
- String
- Boolean
- Class object
- Issue(Class Object)
- Repository(Class Object)
- Milestone(Class object)

- GitHub(Class Object)
- Label(Class Object)
- Figure(Class Object)

## 6.6 List of Special Algorithms

### Metric 1:

IF (item EQUALS "Total Opened Issues"):

```
write_result(tab1, item, len(Total_open_issues))
OUTPUT("Opened issues : {}".format(len(Total_open_issues)))
answers_list.append([item, len(Total_open_issues)])
```

### Metric 2:

ELSEIF (item EQUALS "Total Closed Issues"):

```
write_result(tab2, item, len(Total_closed_issues))
OUTPUT("Closed issues : {}".format(len(Total_closed_issues)))
answers_list.append([item, len(Total_closed_issues)])
```

### Metric 3:

ELSEIF (item EQUALS "Difference between opened and closed issues"):

```
SET temp TO []
SET temp TO abs(len(Total_open_issues)-len(Total_closed_issues))

SET labels TO ["Total open issues: "+str(len(Total_open_issues)), "Total
close issues: "+str(len(Total_closed_issues))]
SET sizes TO [len(Total_open_issues), len(Total_closed_issues)]
draw_pie_chart(tab3, labels, sizes)
OUTPUT("Difference between opened and closed issues : {}".format(temp))
answers_list.append([item, temp])
```

### Metric 4:

ELSEIF (item EQUALS "Distribution score of issues on contributors"):

```
SET contrArray TO []
SET openedIssueUserArray TO []
SET repo_contributors TO Repo.get_contributors()
FOR repo_contributor IN repo_contributors:
    IF (repo_contributor,0) not IN contrArray:
        contrArray.append([repo_contributor,0])
Total_issues=[]
```



```

FOR issue IN Total_issues:
    FOR cont IN contrArray:
        if(cont[0] IN issue.assignees):
            cont[1] += 1
SET sample TO []
FOR cont IN contrArray:
    sample.append(cont[1])

SET stdeviation TO statistics.stdev(sample)
SET meann TO statistics.mean(sample)

SET result TO stdeviation/meann

```

```

SET labels TO []
SET sizes TO []
FOR cont IN contrArray:
    labels.append(str(re.findall("[^"]*" , str(cont[0]))))
    sizes.append(int(cont[1]))
draw_bar_chart(tab4, labels, sizes)
OUTPUT("Distribution score of issues on contributors: {0:.3f}".format(result))
answers_list.append([item, result])

```

#### **Metric 5:**

```

ELSEIF (item EQUALS "Number of label types"):
    SET control1 TO 0
    SET control2 TO 0
    SET defaultLabelSet TO ["bug", "documentation", "duplicate", "enhancement", "good first issue", "help wanted", "invalid", "question", "wontfix"]
    SET labelName TO []
    FOR label IN Repo.get_labels():
        labelName.append(label.name)
    FOR tempLabel IN labelName:
        IF tempLabel not IN defaultLabelSet:
            SET control1 TO control1 + 1
        ELSE:
            SET control2 TO control2 + 1

    SET labels TO ["In Default Set: "+str(control2), "NOT In Default Set: "+str(control1)]
    SET sizes TO [control2, control1]
    draw_pie_chart(tab5, labels, sizes)

    SET labels TO ["Number of label types: "+str(control2+control1)]
    SET sizes TO [control2+control1]
    draw_bar_chart(tab5, labels, sizes)

```

```

        OUTPUT("Number of label types: {}{} of them are IN default set, {} of
them are NOT IN default set.".format(control1+control2, control2, control1))
        answers_list.append([item, control1+control2])

```

### **Metric 6:**

```

ELSEIF (item EQUALS "Label usage frequency"):
    SET labelName TO []
    SET labelCount TO 0

    FOR issue IN Total_issues:
        SET total_labels TO issue.get_labels()
        FOR label IN total_labels:
            if(label != None):
                labelCount += 1
    SET totalIssues TO len(Total_issues)

    SET labels TO ["Used Labels FOR All Issues: "+str(labelCount), "NOT
Used Labels FOR All Issues: "+str(totalIssues - labelCount)]
    SET sizes TO [labelCount, totalIssues - labelCount]
    draw_pie_chart(tab6, labels, sizes)

    OUTPUT("Label usage frequency: %{}{}Total issues {}, labels are used {}
times.".format(int((labelCount/totalIssues)*100),totalIssues,labelCount))
    SET control TO totalIssues/2
    answers_list.append([item, int((labelCount/totalIssues)*100)])

```

### **Metric 7:**

```

ELSEIF (item EQUALS "Total amount of contributors"):
    SET contrArray TO []
    SET openedIssueUserArray TO []
    SET repo_contributors TO Repo.get_contributors()
    FOR repo_contributor IN repo_contributors:
        IF repo_contributor.id not IN contrArray:
            contrArray.append(repo_contributor.id)
    SET contr_num TO len(contrArray)
    IF contr_num > 1:
        OUTPUT("Total amount of contributors: {}".format(contr_num))
    ELSE:
        OUTPUT("{}There is NO contributors FOR this repo yet.")

    write_result(tab7, item, contr_num)
    answers_list.append([item, contr_num])

```

**Metric 8:**

```

ELSEIF (item EQUALS "Total number of assignees on specific Repo"):
    issues=Repo.get_issues()
    SET assignee TO []
    FOR item IN issues:
        SET x TO item.assignees
        FOR member IN x:
            assignee.append(member)
    SET assignee TO len(list(dict.fromkeys(assignee)))
    OUTPUT("Total number of assignee is: {}".format(assignee))

write_result(tab8, item, assignee)
answers_list.append([item, assignee])

```

**Metric 9:**

```

ELSEIF (item EQUALS "Total number of comments"):
    SET open_total_comments TO 0
    SET closed_total_comments TO 0
    FOR issue IN open_issues:
        SET comment_no TO issue.comments
        open_total_comments += comment_no
    SET comment_no TO 0
    FOR issue IN closed_issues:
        comment_no= issue.comments
        closed_total_comments += comment_no
    OUTPUT("Total number of comments: {}".format(open_total_comments +
closed_total_comments))

write_result(tab9, item, open_total_comments + closed_total_comments)
answers_list.append([item, open_total_comments + closed_total_comments])

```

**Metric 10:**

```

ELSEIF (item EQUALS "Avg. Comment Length"):
    SET total TO 0
    SET temp_comment TO 0
    SET count TO 0
    SET labels TO []
    SET sizes TO []
    FOR issue IN Repo.get_issues():
        SET comments TO issue.get_comments()
        SET temp TO 0
        SET counter TO 0
        FOR comment IN comments:

```

```

        FOR character IN comment.body:
            SET temp TO temp + 1

        SET counter TO counter + 1
        labels.append(counter)
        OUTPUT(counter)
        sizes.append(temp)

    IF temp != 0:
        SET temp_comment TO temp_comment + temp/counter
    ELSE:
        SET temp_comment TO 0
    SET count TO count + 1
SET total TO temp_comment/count
OUTPUT("Avg. Comment Lenght: {}".format(total))

draw_scatter_chart(tab10, labels, sizes)
answers_list.append([item, total])

```

#### **Metric 11:**

```

ELSEIF (item EQUALS "Total number of used labels"):
    SET TotalIssues TO []
    SET labelTwins TO []
    FOR issue IN TotalIssues:
        SET labels TO issue.get_labels()
        FOR label IN labels:
            labelTwins.append((label.name))
    OUTPUT("Total number of used labels: {}".format(len(labelTwins)))

write_result(tab11, item, len(labelTwins))
answers_list.append([item, len(labelTwins)])

```

#### **Metric 12:**

```

ELSEIF (item EQUALS "Mean time to response the issues."):
    SET response_time TO []
    SET labels TO []
    SET sizes TO []
    SET Mean_response TO 0
    SET counter TO 1
    FOR issue IN Total_closed_issues:
        created=issue.created_at
        SET updated TO issue.updated_at
        SET closed TO issue.closed_at
        SET response TO updated-created

```

```

    SET duration TO closed – created
    response_time.append(response)
    labels.append(counter)
    counter += 1
FOR x IN response_time:
    SET Mean_response TO Mean_response + x.days
    sizes.append(x.days)
SET Mean_response TO Mean_response/len(response_time)
OUTPUT("Mean time to response the issues: {}".format(Mean_response))

draw_line_chart(tab12, labels, sizes)
answers_list.append([item, Mean_response])

```

### **Metric 13:**

```

ELSEIF (item EQUALS "Total amount of Milestones"):
    SET open_milestones TO Repo.get_milestones(state='open')
    SET close_milestones TO Repo.get_milestones(state='closed')
    open_milestones_list=[]
    SET closed_milestones_list TO []
    FOR milestone IN open_milestones:
        open_milestones_list.append(milestone)
    FOR milestone IN close_milestones:
        closed_milestones_list.append(milestone)
    OUTPUT("Total amount of Milestones: {}".format(len(open_milestones_list)
+ len(closed_milestones_list)))

    SET labels TO ["Total amount of Milestones"]
    SET sizes TO [len(open_milestones_list) + len(closed_milestones_list)]
    draw_bar_chart(tab13, labels, sizes)

    SET labels TO ["Total Opened Milestones: "+str(len(open_milestones_list)),"Total
Closed Milestones: "+str(len(closed_milestones_list))]
    SET sizes TO [len(open_milestones_list), len(closed_milestones_list)]
    draw_pie_chart(tab13, labels, sizes)
    answers_list.append([item, len(open_milestones_list) + len(closed_milestones_list)])

```

### **Metric 14:**

```

ELSEIF (item EQUALS "Total Opened Milestones"):
    SET open_milestones TO Repo.get_milestones(state='open')
    open_milestones_list=[]
    FOR milestone IN open_milestones:
        open_milestones_list.append(milestone)
    OUTPUT("Total Opened Milestones: {}".format(len(open_milestones_list)))

    write_result(tab14, item, len(open_milestones_list))

```

```
answers_list.append([item, len(open_milestones_list)])
```

**Metric 15:**

```
ELSEIF (item EQUALS "Total Closed Milestones"):
    SET close_milestones TO Repo.get_milestones(state='closed')
    SET closed_milestones_list TO []
    FOR milestone IN close_milestones:
        closed_milestones_list.append(milestone)
    OUTPUT("Total Closed Milestones: {}".format(len(closed_milestones_list)))

    write_result(tab15, item, len(closed_milestones_list))
    answers_list.append([item, len(closed_milestones_list)])

ELSE:
    OUTPUT("Error...")
```

# Chapter 7

## Project Management

### 7.1 Software Estimations

#### 7.1.1 Compute Unadjusted Total of Function Points (UTFP)

Table 7.1: Unadjusted Total of Function Points (UTFP)

| Program C.                  | Low Complexity     | Medium Complexity | High Complexity      | Total   |
|-----------------------------|--------------------|-------------------|----------------------|---------|
| Inputs                      | $10 \times 3 = 30$ | $0 \times 4 = 0$  | $0 \times 6 = 0$     | $= 30$  |
| Outputs                     | $0 \times 4 = 0$   | $0 \times 5 = 0$  | $24 \times 7 = 168$  | $= 168$ |
| Inquiries                   | $0 \times 3 = 3$   | $5 \times 4 = 20$ | $15 \times 6 = 90$   | $= 120$ |
| Logical<br>Internal Files   | $0 \times 7 = 0$   | $0 \times 10 = 0$ | $1 \times 15 = 0$    | $= 15$  |
| External<br>Interface Files | $0 \times 5 = 0$   | $0 \times 7 = 0$  | $10 \times 10 = 100$ | $= 100$ |

- Inputs:
  - URL info, appropriate chart type, calculation type, issue metrics, questions are taking from User for Low complexity inputs.
- Outputs:
  - visualised graphs which are 6 type (this number is changeable according to chosen metrics) for High Complexity outputs.
  - Also, added 24 High Complexity outputs for 13 processed issue related metrics and 11 Questions.
  - Those values can be changed in future iterations, according to new added or removed distributed metrics.
- Inquiries:
  - get comments, closed at and created at, id, milestone are taking from GitHub System for Medium Complexity inputs.
  - And the other 15 about issues' info for High Complexity inputs.

- Logical Internal Files: The system has a .csv file in internal approach.
- External Interface:
  - Issue(Class Object), Repository(Class Object), Milestone(Class object), Github(Class Object) etc. are data types which takes from Github API.

Totally, Unadjusted function-point total(UTFP) = 433

### 7.1.2 Computer Value adjustment factor (VAF)

Table 7.2: Total of Degree of Influence (TDI)

| General System Characteristics (GPC) | Degree of Influence(DI)(0-5) |
|--------------------------------------|------------------------------|
| Data Communications                  | 4                            |
| Distributed Processing               | 2                            |
| Performance                          | 3                            |
| Heavily used configuration           | 1                            |
| Transaction Rates                    | 0                            |
| On line data entry                   | 0                            |
| Design for end-user efficiency       | 2                            |
| Online updates                       | 0                            |
| Complex processing                   | 4                            |
| Usable in other applications         | 0                            |
| Installation ease                    | 4                            |
| Operational ease                     | 0                            |
| Multiple sites                       | 0                            |
| Facilitate change                    | 2                            |
| Total of Degree of Influence         | 22                           |

Total of Degree of Influence (TDI) = 22

Value Adjustment Factor (VAF) =  $(TDI * 0.01) + 0.65$

VAF =  $(22 * 0.01) + 0.65 = 0.87$

### 7.1.3 Compute Adjusted Total of Function Points (ATFP)

ATFP =  $433 * 0.87 = 377$

### 7.1.4 Compute Estimated Lines of Code (LOC)

Lines Of Code (LOC) = ATFP \* Language Unit Size

If we look at this website, we could not find Python in the table, so we assumed Object-Oriented default as a value for language unit size.

LOC =  $377 * 29 = 10933$



### 7.1.5 Total Development Time(TDEV)

$$KDSI = ATFP * \text{Language Unit Size}/1000 = \text{LOC}/1000$$

$$KDSI = 10933 / 1000 = 10.933$$

Assume We used semi-detached as Basic COCOMO.

$$\text{Man} - \text{months}(MM) = 3.0 * (10.933)^{1.12} = 43.7$$

$$\text{TotalDevelopmentTime}(TDEV) = 2.5 * (43.7)^{0.35} = 9.37 = 9\text{months}$$

### 7.1.6 Rough Schedule Estimate

For we calculated Rough Schedule Estimate, according to Business and Average in Class. We used Business estimation because we thought our application suits this type of classification and we used average estimation because since we are student and we have another classes we may not develop this application full time so we decided average time suits our group the best.

$$\text{RoughScheduleEstimate} = 433^{0.43} = 13.6 = 14\text{months}$$

### 7.1.7 Efficient Schedule

For Efficient Schedule, we have 10933 LOC ,so approximately 10000 LOC's suitable for us. When we look at 7.1 table, we can say that we have 4.9 months for schedule and 5 man-months for effort. Figure 7.1 shows that estimate the schedule in months and effort in man-months.

|                                | Systems Products     |                            | Business Products    |                            | Shrink-Wrap Products |                            |
|--------------------------------|----------------------|----------------------------|----------------------|----------------------------|----------------------|----------------------------|
| System Size<br>(lines of code) | Schedule<br>(months) | Effort<br>(man-<br>months) | Schedule<br>(months) | Effort<br>(man-<br>months) | Schedule<br>(months) | Effort<br>(man-<br>months) |
| 10,000                         | 8                    | 24                         | 4.9                  | 5                          | 5.9                  | 8                          |
| 15,000                         | 10                   | 38                         | 5.8                  | 8                          | 7                    | 12                         |
| 20,000                         | 11                   | 54                         | 7                    | 11                         | 8                    | 18                         |
| 25,000                         | 12                   | 70                         | 7                    | 14                         | 9                    | 23                         |
| 30,000                         | 13                   | 97                         | 8                    | 20                         | 9                    | 32                         |
| 35,000                         | 14                   | 120                        | 8                    | 24                         | 10                   | 39                         |
| 40,000                         | 15                   | 140                        | 9                    | 30                         | 10                   | 49                         |
| 45,000                         | 16                   | 170                        | 9                    | 34                         | 11                   | 57                         |
| 50,000                         | 16                   | 190                        | 10                   | 40                         | 11                   | 67                         |
| 60,000                         | 18                   | 240                        | 10                   | 49                         | 12                   | 83                         |
| 70,000                         | 19                   | 290                        | 11                   | 61                         | 13                   | 100                        |
| 80,000                         | 20                   | 345                        | 12                   | 71                         | 14                   | 120                        |
| 90,000                         | 21                   | 400                        | 12                   | 82                         | 15                   | 140                        |
| 100,000                        | 22                   | 450                        | 13                   | 93                         | 15                   | 160                        |
| 120,000                        | 23                   | 560                        | 14                   | 115                        | 16                   | 195                        |
| 140,000                        | 25                   | 670                        | 15                   | 140                        | 17                   | 235                        |
| 160,000                        | 26                   | 709                        | 15                   | 160                        | 18                   | 280                        |
| 180,000                        | 28                   | 910                        | 16                   | 190                        | 19                   | 320                        |
| 200,000                        | 29                   | 1,300                      | 17                   | 210                        | 20                   | 360                        |
| 250,000                        | 32                   | 1,300                      | 19                   | 280                        | 22                   | 470                        |
| 300,000                        | 34                   | 1,650                      | 20                   | 345                        | 24                   | 590                        |
| 400,000                        | 38                   | 2,350                      | 22                   | 490                        | 27                   | 830                        |
| 500,000                        | 42                   | 3,100                      | 25                   | 640                        | 29                   | 1,100                      |

Figure 7.1: Efficient Schedules

### 7.1.8 Estimated Team Size

Finally, our estimated team size should be  $\text{Estimated Team Size} = \text{Man-months(MM)} / \text{Total Development Time(TDEV)}$   $43.7 / 10.933 = 3.99 = 4$  members

## 7.2 Milestones, Durations and Task Allocations

| Milestones & Main Tasks                                                 | Duration (Week) | Allocated Member                      |
|-------------------------------------------------------------------------|-----------------|---------------------------------------|
| <b>M1: Graphical User Interface</b>                                     |                 |                                       |
| Investigate tools for GUI                                               | 2               | Çağatay Gültekin, Burak K. Kahraman   |
| Creating the outline of GUI                                             | 2               | İlbey Evcil                           |
| Creation of the specified GUI                                           | 3               | Muhammed Didin, İlbey Evcil           |
| <b>M2: Set up complete pipeline</b>                                     |                 |                                       |
| Review of Github API                                                    | 2               | Everyone                              |
| Learning how issues works                                               | 2               | Everyone                              |
| Determining the pipeline's structure                                    | 2               | İlbey Evcil, Çağatay Gültekin         |
| Receiving data from Github                                              | 1               | Muhammed Didin, Çağatay Gültekin      |
| Outputting basic metrics                                                | 1               | Muhammed Didin                        |
| <b>M3: Calculating the metrics</b>                                      |                 |                                       |
| Determining metrics                                                     | 3               | Everyone                              |
| Investigate related works                                               | 2               | Çağatay Gültekin, Burak Kaan Kahraman |
| Adding distributed metrics                                              | 2               | Muhammed Didin, İlbey Evcil           |
| Completing of main script                                               | 2               | Everyone                              |
| Export data and visualize metrics                                       | 2               | Everyone                              |
| Preparation of demo presentation                                        | 1               | Everyone                              |
| <b>M4: Calculating questions</b>                                        |                 |                                       |
| Determining questions                                                   | 2               | Everyone                              |
| Adding distributed questions                                            | 2               | Burak K. Kahraman, İlbey Evcil        |
| Expanding of main script                                                | 2               | Çağatay Gültekin, Burak K. Kahraman   |
| Export data as a table                                                  | 1               | Muhammed Didin, Çağatay Gültekin      |
| <b>M5: Reworking our GUI with Tkinter</b>                               |                 |                                       |
| Creating the outline of GUI                                             | 2               | Everyone                              |
| Implementing new features to our GUI (such as saving .pickle file etc.) | 2               | İlbey Evcil, Burak Kaan Kahraman      |
| Implementing tooltips and hints to our GUI                              | 2               | Muhammed Didin                        |
| <b>M6: Software Testing</b>                                             |                 |                                       |
| Test plan report                                                        | 2               | Everyone                              |
| GUI and integration test                                                | 2               | Muhammed Didin, İlbey Evcil           |
| Issue-related and question-based metrics test                           | 2               | Çağatay Gültekin, Muhammed Didin      |
| Visualization and export function test                                  | 1               | Burak K. Kahraman, İlbey Evcil        |
| Test result report                                                      | 2               | Everyone                              |
| <b>M7: Researching and Prep. of Demo</b>                                |                 |                                       |
| Investigating on determined repos via our tool                          | 2               | Muhammed Didin, Çağatay Gültekin      |
| Collecting and interpreting these results                               | 2               | Burak Kaan Kahraman, İlbey Evcil      |
| Preparation of final report                                             | 1               | Everyone                              |
| Preparation of final demo and presentation                              | 1               | Everyone                              |

## 7.3 Gantt Chart

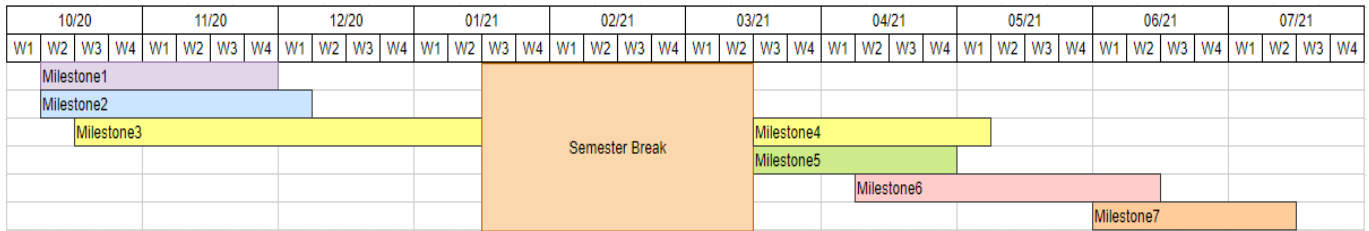


Figure 7.2: Gantt Chart

Figure 7.2 shows that we divide the works we plan to do during this period into the milestones and show who will complete the tasks in each milestone in what time. We can calculate how long each task takes by looking at the volume it occupies in the horizontal area. In addition, the interdependence of each task can be seen with the line between them.

# Chapter 8

## Software Implementation

Python programming language is used in Software Implementation. This programming language has been preferred because it has a library that communicates with GitHub, its analysis tools are advanced and it is easy to use.

### 8.1 List of Libraries

#### 8.1.1 Tkinter

While developing the Graphical User Interface, the Tkinter library of the Python language was used. This library has been preferred because it can be accessed from the code and is easy to use while developing a desktop application. During GUI development, Label, Button, TextField, ListView, File Explorer, Tool Tip components from this library were used.

#### 8.1.2 PyGithub

This library is used to communicate with the GitHub API. It is necessary to send the URL of a repository to contact with GitHub API through library. At the end of the operation, the library returns a repository object. This object contains the information required for analysis.

#### 8.1.3 matplotlib

The matplotlib library derives many types of graphs from data in different data types and data structures. We also used various graphics such as bar chart, box plot, line graph in our project. We selected and applied the most suitable graphical representation for our data from the graphics in the matplotlib library.

#### 8.1.4 csv

The csv library takes the derived or existing data and converts it into a .csv file in the desired format and enables the data to be exported from the program. In our project, we also created .csv file and exported the metrics and Question-Answer table we calculated via csv library.

### 8.1.5 pickle

In order to overcome the GitHub API Rate Limit problem, repository objects are kept on the local computer. The pickle library was used during this process. With this library, repository objects can be saved to the computer in binary format and then read multiple times if desired. In this way, GitHub Rate Limit problem has been overcome to some extent, since no saving is performed for every read operation.

Other libraries used while developing the application are listed below.

- statistics
- sys
- os
- re
- PIL
- urllib
- io

# Chapter 9

## Software Testing

### 9.1 Testing Strategy

While doing the tests that we planned and reported in our Test Plan report, we tested some parts manually, while the rest were automated with the tests we wrote. Considering the problems that we may encounter while testing our tool, a wide test case scenario was prepared. Each of these test case scenarios was tested one by one. As a result of the tests, our tool has successfully passed every test case we have prepared. Apart from these, although our tool also passed the Performance test, it could not fulfill the desired tasks at some points in the Stress test due to the GitHub API Rate Limit.

We have successfully performed all the tests that we specified in our test plan. We conducted these tests on repositories with the different number of issues and attributes. In most of our tests, we got the results we wanted and expected. In the stress testing part, we measured our tool's resistance to API rate limits with various repositories and various situations. The results in stress testing did not fully meet our expectations.

In most tests, we used our own test repository. The reference photo 9.1 of the issue part of our test repository is below:

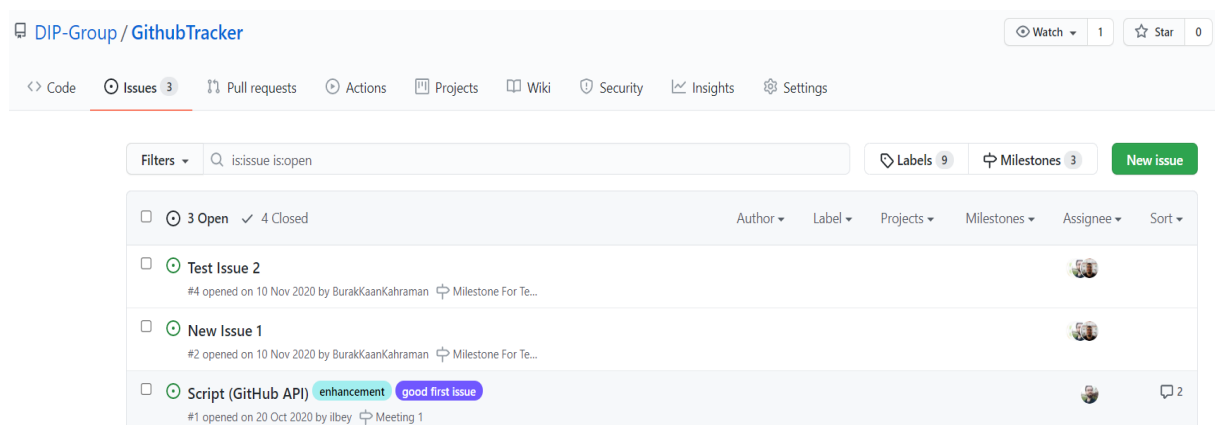


Figure 9.1: DIP-Group/GithubTracker repository issues page

## 9.2 Unit Testing

### 9.2.1 Graphical User Interface

#### Participants

- Çağatay Gültekin
- İlbey Evcil

#### Pass/Fail Criteria

Graphical User Interface testing was carried out manually. It is checked whether the program performs the required operation on the terminal or directly on the graphical user interface. If the graphical user interface has done the required operation, the test passed, if not, it failed.

#### Methodology

Our developed tool has a graphical user interface for the user to make selections. We performed the test of our GUI unit manually. We checked whether the URL entered on the GUI or the selected metrics and questions were received correctly from the user, by printing them on the terminal after they were received.

### 9.2.2 Metric Calculations

(Total Opened Issues, Total Closed Issues, Difference between opened and closed issues, Distribution of issues on contributors, Number of label types, Label usage frequency, Total amount of contributors, Total number of assignee on specific repository, Total number of comments, Avg. Comment Length, Label colors for default Specific titles, Mean time to response the issues, Total amount of Milestones, Total Opened Milestones, Total Closed Milestones)

#### Participants

- İlbey Evcil
- Burak Kaan Kahraman

#### Pass/Fail Criteria

Figure 9.2 is shown in which cases the outputs of our metric functions are pass/fail.



| Metrics                                      | Pass Condition               | Fail Condition                             |
|----------------------------------------------|------------------------------|--------------------------------------------|
| Total Opened Issues                          | Response(3), Type(Integer)   | No response, Wrong Value, Wrong Type, Null |
| Total Closed Issues                          | Response(6), Type(Integer)   | No response, Wrong Value, Wrong Type, Null |
| Difference between opened and closed issues  | Response(3), Type(Integer)   | No response, Wrong Value, Wrong Type, Null |
| Distribution score of issues on contributors | Response(0.165), Type(Float) | No response, Wrong Value, Wrong Type, Null |
| Number of label types                        | Response(9), Type(Integer)   | No response, Wrong Value, Wrong Type, Null |
| Label usage frequency                        | Response(22), Type(Integer)  | No response, Wrong Value, Wrong Type, Null |
| Total amount of contributors                 | Response(4), Type(Integer)   | No response, Wrong Value, Wrong Type, Null |
| Total number of assignees on specific Repo   | Response(4), Type(Integer)   | No response, Wrong Value, Wrong Type, Null |
| Total number of comments                     | Response(2), Type(Integer)   | No response, Wrong Value, Wrong Type, Null |
| Avg. Comment Length                          | Response(6.67), Type(Float)  | No response, Wrong Value, Wrong Type, Null |
| Total number of used labels                  | Response(2), Type(Integer)   | No response, Wrong Value, Wrong Type, Null |
| Mean time to response the issues             | Response(52.17), Type(Float) | No response, Wrong Value, Wrong Type, Null |
| Total amount of Milestones                   | Response(4), Type(Integer)   | No response, Wrong Value, Wrong Type, Null |
| Total Opened Milestones                      | Response(3), Type(Integer)   | No response, Wrong Value, Wrong Type, Null |
| Total Closed Milestones                      | Response(1), Type(Integer)   | No response, Wrong Value, Wrong Type, Null |

Figure 9.2: Unit Test Conditions for Metric Calculations

## Methodology

This unit is for functions where our tool calculates certain metrics. We used a function for each metric calculation. While testing our metric unit, we checked whether each of these functions passed the criteria we had determined. We performed this check automatically thanks to a script we patched. We were not able to run our tests using the "pytest" library as we specified in the test plan. Because we discovered that the way of use is a little different than we thought. Instead of this library, we created our script ourselves using the same mentality. We tested our functions within the scope of this unit on a GitHub repository that we have created before. Therefore, we were able to calculate what the outputs of each of our functions were and easily check whether the functions gave these outputs or not, thanks to this script we wrote. The features we control; The results of the functions for this test repository were checked whether they were "Null", their types and values. In other words, as a result, our repository manually calculated what outputs should come from each function, and whether these functions gave these results was automatically tested using this script that you can see in Figure 9.3. You can also see how we used in main function in Figure 9.4

```

def Test_Function(index,control_type,value):
    global answers_list

    item=answers_list[index]
    item=item[1]

    if(item):
        if(type(item)==control_type):
            if(item==value):
                pass
            else:
                return False
        else:
            return False
    else:
        return False
    return True

```

Figure 9.3: Example from script used in metric calculations

```

if __name__ == "__main__":
    """
    systemTest()
    systemTest2()
    get_repo_test("https://github.com/DIP-Group/GithubTracker")
    """
    repo=load_repo_from_file("GithubTracker.pkl")

    #3,6,3,0.1649572197684645,9,22,4,4,2,6.666666666666667,2,52.166666666666664,4,3,1
    testResult=[]
    calculate_metrics(repo)
    testResult.append(Test_Function(0,int,3))
    testResult.append(Test_Function(1,int,6))
    testResult.append(Test_Function(2,int,3))
    testResult.append(Test_Function(3,float,0.1649572197684645))
    testResult.append(Test_Function(4,int,9))
    testResult.append(Test_Function(5,int,22))
    testResult.append(Test_Function(6,int,4))
    testResult.append(Test_Function(7,int,4))
    testResult.append(Test_Function(8,int,2))
    testResult.append(Test_Function(9,float,6.666666666666667))
    testResult.append(Test_Function(10,int,2))
    testResult.append(Test_Function(11,float,52.166666666666664))
    testResult.append(Test_Function(12,int,4))
    testResult.append(Test_Function(13,int,3))
    testResult.append(Test_Function(14,int,1))

    counter = 1
    for element in testResult:
        print("Metric Calculation {}:{}".format(counter))
        if(element):
            print("Test Success")
        else:
            print("Test Failed")
        counter += 1

```

Figure 9.4: Example from script used in metric calculations

## 9.3 Exporting Data to .csv Format

### 9.3.1 Participants

- Muhammed Didin
- Çağatay Gültekin

### 9.3.2 Pass/Fail Criteria

- The tests are failed, if
  - there is a NULL value,
  - the result does not match with the GitHub platform,
  - more than one repository is used, the results are mixed and not written one after the other.
- The tests are passed, if
  - there is no NULL value,
  - the result match with the GitHub platform,
  - more than one repository is used, the results are not mixed and written one after the other.

### 9.3.3 Methodology

An operation performed by our tool is to present the data to be analyzed later to the user in .csv format. If the user wishes, s/he can download the data in .csv format to her own computer by using the export button on the calculation screen. A manual test was carried out to realize this test. GitHub platform and the file that came out as a result of our application were compared by the fact that the program does not give empty results, gives correct results - and if there is more than one repository- these results are written one after the other without any conflicts.

## 9.4 Exporting Data as Graphical Representations

### 9.4.1 Participants

- Muhammed Didin
- Çağatay Gültekin

### 9.4.2 Pass/Fail Criteria

- The tests are failed, if
  - there is a empty result screen,
  - wrong graphical representation used,
  - the item that shouldn't represented as a graph is represented as graph,

the result does not match with the GitHub platform,

- The tests are passed, if  
appropriate graphical representation used,  
the result match with the GitHub platform.

### 9.4.3 Methodology

One operation performed by our tool is to fetch the data to be analyzed from Github and plot an appropriate graph about the particular metric for the user. The main purpose of doing this is to enable the user to easily understand the data obtained and compare them. In this unit, we manually tested both the matching of the data in Github and the data in the graph, drawing the appropriate type of graph, and making the metric calculation correctly and smoothly.

## 9.5 Answers to the questions

(Are labels used for issue management?, Is the default set of labels sufficient for issue management?, Is every contributor active in issue management?, Is the responsibility of opening and closing issues equally distributed among contributors?, Are the lifetimes of the issues consistent?, Are the comments made by the contributors equally distributed among contributors?, Is commenting used consistently in all issues?, Are the issues with assignee completed earlier to compared the ones without assignee?,What is the ratio of the opened issues grouped under milestone and those not grouped?, What does the average time between opening and closing the issue?,What does the average time between opening and closing the milestone?, Are the comments be posted after the issues are closed or during the process?, Are the issues grouped under milestone completed earlier compared to the ones not grouped?)

### 9.5.1 Participants

- İlbey Evcil
- Burak Kaan Kahraman

### 9.5.2 Pass/Fail Criteria

Figure 9.5 is shown in which cases the outputs of our "Answers to the questions" functions are pass/fail.

| Questions                                                                                   | Pass Condition              | Fail Condition                             |
|---------------------------------------------------------------------------------------------|-----------------------------|--------------------------------------------|
| Are labels used for issue management?                                                       | Response(No) Type(String)   | No response, Wrong Value, Wrong Type, Null |
| Is the default set of labels sufficient for issue management?                               | Response(Yes) Type(String)  | No response, Wrong Value, Wrong Type, Null |
| Is every contributor active in issue management?                                            | Response(No) Type(String)   | No response, Wrong Value, Wrong Type, Null |
| Is the responsibility of opening and closing issues equally distributed among contributors? | Response(No) Type(String)   | No response, Wrong Value, Wrong Type, Null |
| Are the life times of the issues consistent?                                                | Response(No) Type(String)   | No response, Wrong Value, Wrong Type, Null |
| Are the comments made by the contributors equally distributed among contributors?           | Response(No) Type(String)   | No response, Wrong Value, Wrong Type, Null |
| Is commenting used consistently in all issues?                                              | Response(No) Type(String)   | No response, Wrong Value, Wrong Type, Null |
| Are the issues with assignee completed earlier to compared the ones without assignee?       | Response(No) Type(String)   | No response, Wrong Value, Wrong Type, Null |
| What is the ratio of the opened issues grouped under milestone and those not grouped?       | Response(100) Type(Integer) | No response, Wrong Value, Wrong Type, Null |
| What does the average time between opening and closing the issue?                           | Response(36.0) Type(Float)  | No response, Wrong Value, Wrong Type, Null |
| What does the average time between opening and closing the milestone?                       | Response(7.0) Type(Float)   | No response, Wrong Value, Wrong Type, Null |
| Are the comments be posted after the issues are closed or during the process?               | Response(No) Type(String)   | No response, Wrong Value, Wrong Type, Null |
| Are the issues grouped under milestone completed earlier compared to the ones not grouped?  | Response(Yes) Type(String)  | No response, Wrong Value, Wrong Type, Null |

Figure 9.5: Unit Test Conditions for Answer to the Questions

### 9.5.3 Methodology

Another task of our tool, is to give answers to the research questions we have determined as a result of the calculations it has made for a given specific repository. We used a function for each question calculation. While testing our question unit, we checked whether each of these functions passed the criteria we had determined. We performed this control automatically, thanks to a script we made. We tested our functions within the scope of this unit on a Github repository that we have created before. Therefore, we were able to calculate the outputs of each of our functions and easily check whether the functions give these outputs or not, thanks to this script we wrote. The features we control; The results of the functions for this test repo were checked whether they were "Null", their types, and values. In other words, as a result, our repository manually calculated what outputs should come from each function, and whether these functions gave these results was automatically tested using this script. You can see in Figure 9.6 that how we implement test function in our script. Also, you can see in Figure 9.7 that how we implement in main function.

```

def Test_Function(index,control_type,value):
    global answers_list

    item=answers_list[index]
    item=item[1]

    if(item):
        if(type(item)==control_type):
            if(item==value):
                pass
            else:
                return False
        else:
            return False
    else:
        return False
    return True

```

Figure 9.6: Question test function from script used in test

```

if __name__ == "__main__":
    get_repo_test("https://github.com/DIP-Group/GithubTracker")

    repo=load_repo_from_file("GithubTracker.pkl")

    testResult=[]
    calculate_questions(repo)
    testResult.append(Test_Function(0,str,"No"))
    testResult.append(Test_Function(1,str,"Yes"))
    testResult.append(Test_Function(2,str,"No"))
    testResult.append(Test_Function(3,str,"No"))
    testResult.append(Test_Function(4,str,"No"))
    testResult.append(Test_Function(5,str,"No"))
    testResult.append(Test_Function(6,str,"No"))
    testResult.append(Test_Function(7,str,"No"))
    testResult.append(Test_Function(8,int,100))
    testResult.append(Test_Function(9,float,36.0))
    testResult.append(Test_Function(10,float,7.0))
    testResult.append(Test_Function(11,str,"No"))
    testResult.append(Test_Function(12,str,"Yes"))

    counter = 1
    for element in testResult:
        print("Question Calculation {}: ".format(counter))
        if(element):
            print("Test Success")
        else:
            print("Test Failed")
        counter += 1

```

Figure 9.7: Main function from script used in test

## 9.6 System and Integration Testing

### 9.6.1 Participants

- İlbey Evcil
- Burak Kaan Kahraman

## 9.7 Integration Testing

### Methodology

In the integration testing part, we tested whether the connection that we tried to provide with the GitHub API of our tool and the "pygithub" library was realized. We performed this test as automated with the help of a script we wrote. Our script takes any repository URL to perform the test and checks whether it can pull this repository from this URL using the "pygithub" library. If the repository can be pulled, our integration test will be successful and the retrieved repository is saved as a pickle file in the folder it is in. If the specified repository is detected from the system, it is predicted that there may be a problem in connecting with the GitHub API and our integration test will be a failure.

### 9.7.1 System Testing

#### Methodology

Testing of our system is performed using our '.csv' file, which is the final output of our tool. Our tool gives us two '.csv' files output if all options are selected via GUI. We test these files automatically with the help of a script we have written. We observed that our '.csv' file is "Null" if there is any problem anywhere in our system. Thus, by using this feature of our program, we were able to control our entire system. For example, if all metrics are selected in the GUI in our system, but "Null" expressions are observed in between, we can make inferences such as the problem in the GUI, the functions where the calculations are made, the connection established with the Github API, or our function that performs our '.csv' output. As a result, with the help of this script, we can check whether our system is working correctly, and if not, with the help of our other unit tests, we can determine where the problem originates from.

## 9.8 Performance Load and Stress Testing

### 9.8.1 Participants

- Çağatay Gültekin
- Muhammed Didin

### 9.8.2 Performance Load Testing

We conducted a performance load test to measure the working speed of our tool in repositories with different issues. We used repositories below while performing this test. In

each repository, each metric was tried one by one and the elapsed time was found while calculating that metric. The criterion for success when running this test was to take less than 4 seconds on Medium-Large repositories (150 issues). As a result of our tests, it was seen that our tool was able to reach the desired target and thus our system successfully completed this test. At the end of this, the average time for each repository was recalculated in seconds. This time is assumed as the elapsed time for the repository. As a result, the following graph on Figure 9.9 was obtained.



Repositories that is used:

- DIP/GitHubTrackerTestResults: 1 issue  
<https://github.com/DIP-Group/GitHubTrackerTestResults>
- DIP/GitHubTracker: 7 issues  
<https://github.com/DIP-Group/GithubTracker>
- Fnajib: 80 issues  
<https://github.com/FaisalNajibAbdullah/FNajibProyek3>
- Django Datatable View: 169 issues  
<https://github.com/pivotal-energy-solutions/django-datable-view>
- Slack: 698  
<https://github.com/integrations/slack>
- Lombok: 2502  
<https://github.com/projectlombok/lombok>

Please note: The Lombok repository gave an uncertain result as it faced API rate limit issue on most metrics. Therefore, it is not included in the chart. It has been observed that the API Rate limit problem encountered even when only one metric is taken in large repositories in terms of the number of Issues. In these repositories the relevant metric times are accepted as NULL values and are not included when calculating the average result.

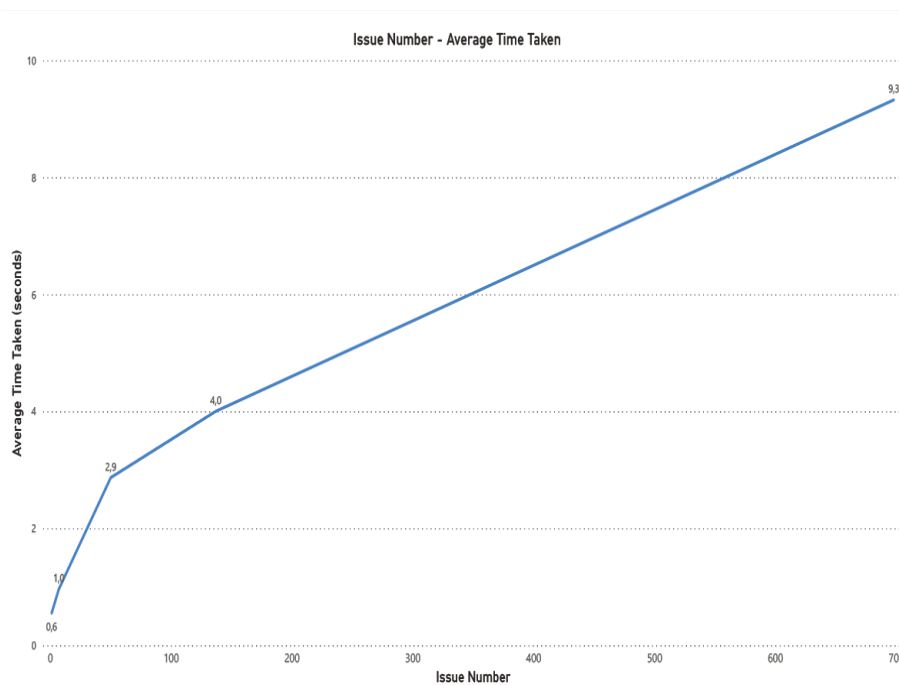


Figure 9.8: Performance Test Results

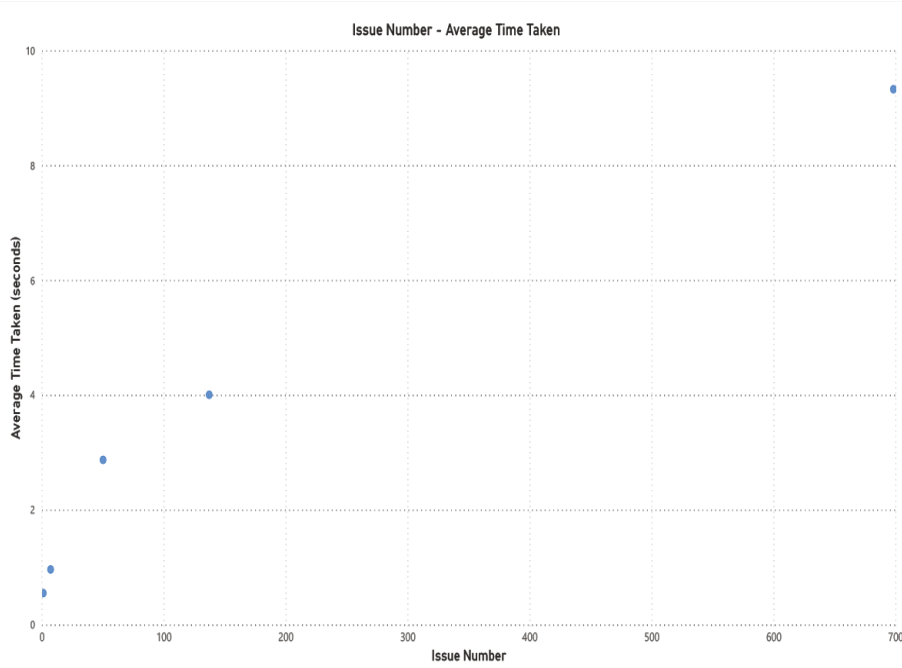


Figure 9.9: Performance Test Results

### 9.8.3 Stress Testing

In this stress testing we have done, we tried to measure how long it works until the API rate given to us by GitHub reaches the limit in our application. First, we asked our program to calculate all our metrics for each repository. In this test, it worked last in the repository, which has twenty issues. We then asked our program to calculate all our questions for each repository. In this test, in the repository, which has seven issues, the API rate worked without reaching the limit. Afterward, we made the program calculate 1 metric for each repository. We used our "Distribution of issues on contributors" metric for this transaction because it was the metric that used the most requests for us. Finally, it completed this process in the repository with 698 issues before the API rate reached the limit. Finally, we asked the program to calculate 1 question. We used our "Are labels used for issue management?" question because there was no request difference between the questions, so any question could be used. This process was finally completed before the API rate reached the limit in the repository with 50 issues. The results of these tests are shown in Figure 10.5 as a table.

Repositories that used:

- DIP/GitHubTrackerTestResults: 1 issue  
<https://github.com/DIP-Group/GitHubTrackerTestResults>
- DIP/GitHubTracker: 7 issues  
<https://github.com/DIP-Group/GithubTracker>
- scochran31/README-Creator: 20 issues  
<https://github.com/sochran31/README-Creator>
- kios:50 issues  
<https://github.com/kerberos-io/kios>
- Fnajib: 80 issues  
<https://github.com/FaisalNajibAbdullah/FNajibProyek3>
- Django Datatable View: 169 issues  
<https://github.com/pivotal-energy-solutions/django-datable-view>
- Slack: 698 issues  
<https://github.com/integrations/slack>
- Lombok: 2502 issues  
<https://github.com/projectlombok/lombok>

| Test 1: All calculations doing at the same time               |                             |                             |
|---------------------------------------------------------------|-----------------------------|-----------------------------|
| Num. Issues                                                   | Metrics                     | Questions                   |
| 1                                                             | TRUE                        | TRUE                        |
| 7                                                             | TRUE                        | TRUE                        |
| 20                                                            | TRUE                        | FALSE                       |
| 50                                                            | FALSE<br>(first 5 finished) | FALSE<br>(first 3 finished) |
| Test 2: Just 1 metric or question calculated at the same time |                             |                             |
| Num. Issues                                                   | Metrics                     | Questions                   |
| 1                                                             | TRUE                        | TRUE                        |
| 7                                                             | TRUE                        | TRUE                        |
| 50                                                            | TRUE                        | TRUE                        |
| 80                                                            | TRUE                        | FALSE                       |
| 137                                                           | TRUE                        | FALSE                       |
| 169                                                           | TRUE                        | FALSE                       |
| 698                                                           | TRUE                        | FALSE                       |
| 2502                                                          | FALSE                       | FALSE                       |

Figure 9.10: Stress Test Results

# Chapter 10

## Conclusion

Discovering Issue Tracking Profiles of Software Development Teams (DIP) is an important project aimed to increase the productivity of project managers and coordinators by using issue tracking methodology. It is also a project that researchers who do research on issue tracking can benefit from. This project tries to show that you can obtain important parameters and information with the metrics and questions created from issue tracking data. At the same time, this project is the application of some mathematical and statistical formulas about the issue tracking system, which was made earlier.

In the scope of this project, we aim to develop a tool in order to guide the software developers who use the issue tracking feature in version control systems and to make them use better. Thanks to this tool, large companies and individuals who use this system more professionally are researched, as a result this process, the use of the issue tracking system is understood and the best practice is determined as a result of the analyzes made using these data. Thus, as a result of our analysis, we aim to develop a system that will be a guide for those who will use such platforms for the first time or those who use them incorrectly.

While planning this project, we aimed to obtain a program that can communicate with GitHub API, calculate certain metrics, answer certain questions, obtain certain graphs using these metrics and answers, and export them in .csv format by the end of this period. In addition, we aimed to develop a Graphical User Interface to this program.

Considering our busy schedule during the term, we thought that if we didn't share the tasks, we might have problems with time management, so we could use our time more efficiently by sharing tasks within a schedule. In addition, we increased the interest of all group members in the project by sharing the works by taking sections on every subject such as development and documentation.

Since none of us had experienced the programs used in this project (LaTeX, Github, Trello, TeamGantt etc.) before, we had to spend time learning these applications and improving ourselves in these applications. But at the current stage, we have been able to better integrate the facilities provided by these applications into our project by experiencing and learning these applications.

We spent time to research mathematical or statistical formulas, graphs, etc. that we need in our project. We increased our knowledge about the project topic by sharing research-based project articles, similar statistical researches and related articles that use mathematical formulas with each other and researching similar ones.

While we were developing our project, we had the opportunity to learn how we can improve ourselves and our project by receiving feedback from our academic advisor.

While developing the program, we made great efforts to comply with good software development methodologies.

When any of the group members faced a problem, we tried to find a solution to the problem by coming together with the other group members.

By adding a modular structure to the program we have written, we have increased the readability of the code and reduced the complexity. Our functions are enabled to work for any metric we create. The application we developed has been updated to receive data from multiple repositories. In addition, the graphics are dynamically created according to the determined metrics.

Large companies that are used professionally in the Issue Tracking system will be identified and researches will be conducted using this tool which we developed on them. The data obtained from these companies can be grouped by applying clustering. Thus, better and more accurate, best practice can be determined. These studies will be done in collaboration with Manchester University.

The results of the research will be collected in an article and shared as open-source in order to benefit those concerned.

## 10.1 Example Research Results

Below are our look-up tables, on which the names of repository, Metric and Questions are written. Using the repositories we have shown below, we have made a small-scale analysis example using the metrics and questions in our tool. While doing this sample test on the basis of metric calculation, we took the important repository of various important companies as an example. You can find the details by looking at the tables below.

| Repository Number | Repository Name            |
|-------------------|----------------------------|
| Repo1             | microsoft/api-guidelines   |
| Repo2             | oracle/oracle-db-examples  |
| Repo3             | facebook/fbshipit          |
| Repo4             | microsoft/ML-For-Beginners |
| Repo5             | JetBrains/JWM              |
| Repo6             | JetBrains/JetBrainsRuntime |

Figure 10.1: Repository Names

| Metric Number | Metric Name                                  |
|---------------|----------------------------------------------|
| M1            | Total Opened Issues                          |
| M2            | Total Closed Issues                          |
| M3            | Difference between opened and closed issues  |
| M4            | Distribution score of issues on contributors |
| M5            | Number of label types                        |
| M6            | Label usage frequency                        |
| M7            | Total amount of contributors                 |
| M8            | Total number of assignees on specific Repo   |
| M9            | Total number of comments                     |
| M10           | Avg. Comment Length                          |
| M11           | Total number of used labels                  |
| M12           | Mean time to response the issues             |
| M13           | Total amount of Milestones                   |
| M14           | Total Opened Milestones                      |
| M15           | Total Closed Milestones                      |

Figure 10.2: Metric Names

|       | M1 | M2  | M3  | M4   | M5   | M6   | M7  | M8   | M9    | M10   | M11  | M12   | M13 | M14 | M15 |
|-------|----|-----|-----|------|------|------|-----|------|-------|-------|------|-------|-----|-----|-----|
| Repo1 | 58 | 184 | 126 | 4.93 | 9    | NULL | 53  | 3    | 566   | 74.35 | NULL | 74.09 | 0   | 0   | 0   |
| Repo2 | 7  | 155 | 148 | 2.11 | 12   | NULL | 24  | NULL | 150   | 36.2  | NULL | 32.92 | 0   | 0   | 0   |
| Repo3 | 10 | 110 | 100 | 7.72 | NULL | NULL | 181 | NULL | 0.177 | NULL  | 0    | 108.8 | 0   | 0   | 0   |
| Repo4 | 7  | 78  | 71  | 2.92 | 16   | NULL | 21  | NULL | 0     | 121   | NULL | 4.91  | 0   | 0   | 0   |
| Repo5 | 9  | 21  | 12  | 1    | 9    | 0    | 6   | NULL | 2     | 20    | NULL | NULL  | 0   | 0   | 0   |
| Repo6 | 38 | 241 | 203 | 5.74 | 9    | NULL | 33  | NULL | 0     | 465   | NULL | NULL  | 0   | 0   | 0   |

Figure 10.3: Metrics Result Table

| Question Number | Question Names                                                                              |
|-----------------|---------------------------------------------------------------------------------------------|
| Q1              | Are labels used for issue management?                                                       |
| Q2              | Is the default set of labels sufficient for issue management?                               |
| Q3              | Is every contributor active in issue management?                                            |
| Q4              | Is the responsibility of opening and closing issues equally distributed among contributors? |
| Q5              | Are the life times of the issues consistent?                                                |
| Q6              | Are the comments made by the contributors equally distributed among contributors?           |
| Q7              | Is commenting used consistently in all issues?                                              |
| Q8              | Are the issues with assignee completed earlier to compared the ones without assignee?       |
| Q9              | What is the ratio of the opened issues grouped under milestone and those not grouped?       |
| Q10             | What does the average time between opening and closing the issue?                           |
| Q11             | What does the average time between opening and closing the milestone?                       |
| Q12             | Are the comments be posted after the issues are closed or during the process?               |
| Q13             | Are the issues grouped under milestone completed earlier compared to the ones not grouped?  |

Figure 10.4: Question Names

|       | Q1   | Q2   | Q3  | Q4   | Q5  | Q6 | Q7   | Q8  | Q9  | Q10   | Q11 | Q12 | Q13 |
|-------|------|------|-----|------|-----|----|------|-----|-----|-------|-----|-----|-----|
| Repo1 | NULL | No   | Yes | No   | No  | No | NULL | Yes | 0.0 | 68.08 | 0.0 | No  | No  |
| Repo2 | NULL | No   | No  | NULL | No  | No | No   | No  | 0.0 | 14.06 | 0.0 | No  | No  |
| Repo3 | NULL | NULL | No  | No   | No  | No | No   | Yes | 0.0 | NULL  | 0.0 | No  | No  |
| Repo4 | Yes  | No   | No  | No   | Yes | No | No   | No  | 0.0 | 4.87  | 0.0 | No  | No  |
| Repo5 | No   | Yes  | Yes | No   | Yes | No | No   | Yes | 0.0 | NULL  | 0.0 | No  | No  |

Figure 10.5: Question Results Table

# Bibliography

- [1] Daniel Dietsch, Andreas Podelski, Jaechang Nam, Pantelis Papadopoulos, and Martin Schäf. Monitoring student activity in collaborative software development. 05 2013.
- [2] Sukru Eraslan, Kamilla Kopec-Harding, Caroline Jay, Suzanne M Embury, Robert Haines, Julio César Cortés Ríos, and Peter Crowther. Integrating gitlab metrics into coursework consultation sessions in a software engineering course. *Journal of Systems and Software*, page 110613, 2020.
- [3] J. Kay, N. Maisonneuve, and K. Yacef. Mining patterns of events in students’ team-work data. 2006.
- [4] Megha Mittal and Ashish Sureka. Process mining software repositories from student projects in an undergraduate software engineering course. page 344–353, 2014.
- [5] Faisal A.T. Putra, Harry Budi Santoso, and Rizal Fathoni Aji. Evaluation of learning analytics metrics and dashboard in a software engineering project course. *Global Journal of Engineering Education*, 20(3):171–180, January 2018.